

RELATÓRIO TÉCNICO – ADMINISTRAÇÃO DE DADOS COM POSTGRESQL, MONGODB E QDRANT

1. Introdução

A presente atividade teve como objetivo aplicar conceitos de administração de bancos de dados em uma arquitetura moderna voltada para aplicações de Inteligência Artificial. O trabalho envolveu a utilização integrada de diferentes tecnologias de armazenamento de dados, considerando aspectos relacionados a desempenho, organização, escalabilidade, segurança, governança da informação e recuperação inteligente de dados.

Foram utilizados três tipos de bancos de dados com finalidades distintas:

- **PostgreSQL** para armazenamento estruturado e processamento transacional;
- **MongoDB** para armazenamento de dados semiestruturados em formato documental;
- **Qdrant** para armazenamento vetorial e realização de buscas semânticas baseadas em embeddings.

Durante a execução da atividade, também foi utilizada IA generativa como ferramenta de apoio para análise técnica, identificação de oportunidades de melhoria e proposição de soluções, mantendo a validação final das decisões sob responsabilidade do analista.

2. Implementação do PostgreSQL

2.1 Modelagem Relacional

Foi criado um banco de dados denominado **ua4**, composto pelas seguintes tabelas:

- **customers**
- **products**
- **orders**
- **order_items**

A modelagem utilizou chaves primárias e chaves estrangeiras para garantir integridade referencial entre as entidades.

A tabela **customers** armazena informações dos clientes, a tabela **products** contém os produtos cadastrados, a tabela **orders** registra os pedidos realizados e a tabela **order_items** armazena os itens pertencentes a cada pedido.

Essa estrutura possibilita consultas relacionais eficientes por meio da utilização de JOINS, além de fornecer uma base consistente para análises futuras e integração com aplicações de Inteligência Artificial.

2.2 Inserção e Validação dos Dados

Foram inseridos registros de teste simulando clientes, produtos e pedidos. Após a carga inicial, foram realizados testes de consistência para validar a integridade dos relacionamentos definidos no modelo relacional.

3. Análise e Otimização de Desempenho

3.1 Consulta Avaliada

Foi analisada uma consulta destinada a identificar os produtos com maior faturamento nos últimos 30 dias, considerando apenas pedidos com status de pagamento confirmado.

A consulta realiza:

- filtro por status do pedido;
- filtro por período;
- junções entre as tabelas `orders`, `order_items` e `products`;
- agrupamento por produto;
- ordenação por valor total faturado.

3.2 Avaliação com EXPLAIN ANALYZE

Para avaliar o desempenho da consulta foi utilizado o comando **EXPLAIN ANALYZE**, que permite visualizar o plano de execução gerado pelo PostgreSQL.

A análise inicial demonstrou que a consulta realizava operações de leitura e junção sem o suporte de índices específicos para os campos mais utilizados nos filtros e relacionamentos, indicando oportunidades de otimização.

Embora o ambiente de testes apresentasse pequeno volume de dados, a análise evidenciou pontos que poderiam impactar significativamente o desempenho em ambientes produtivos.

3.3 Otimizações Aplicadas

Foram criados os seguintes índices:

- **idx_orders_status_date** sobre os campos `status` e `order_date`;
- **idx_order_items_order** sobre o campo `order_id`;
- **idx_order_items_product** sobre o campo `product_id`.

Esses índices foram definidos com base nos filtros, junções e agrupamentos utilizados pela consulta.

3.4 Resultados Obtidos

Após a implementação dos índices, a consulta foi executada novamente utilizando **EXPLAIN ANALYZE**.

Observou-se redução do custo estimado do plano de execução de aproximadamente **50 para 29 pontos**, indicando que o otimizador do PostgreSQL passou a utilizar caminhos de acesso mais eficientes aos dados.

Em relação ao tempo real de execução, a redução observada foi de aproximadamente **0,079 ms para 0,067 ms**. Embora o ganho tenha sido pequeno devido ao volume reduzido de registros, os resultados demonstram o potencial de escalabilidade da solução em ambientes com milhares ou milhões de registros.

Os testes confirmaram a importância da análise de planos de execução e da utilização adequada de índices para otimização de consultas em bancos relacionais.

4. Implementação do MongoDB

4.1 Estrutura Documental

Foi criado o banco de dados **reviews_db** contendo a coleção **reviews**, destinada ao armazenamento de avaliações de produtos realizadas por clientes.

Cada documento contém os seguintes campos:

- **product_id**
- **customer**
- **rating**
- **review_text**

Essa estrutura demonstra a flexibilidade do modelo documental para armazenamento de dados semiestruturados, especialmente em cenários que envolvem conteúdo textual variável.

4.2 Inserção dos Documentos

Foram inseridos documentos de teste contendo avaliações de diferentes produtos.

As avaliações armazenam simultaneamente informações estruturadas e comentários livres, característica comum em plataformas de comércio eletrônico e sistemas de análise de sentimento.

4.3 Criação de Índices

Foram criados índices nos campos:

- **product_id**
- **rating**

Esses índices foram implementados para otimizar consultas relacionadas à identificação de produtos e análises de avaliações.

A utilização dos índices reduz a necessidade de varreduras completas da coleção (*Collection Scan*), contribuindo para melhor desempenho e escalabilidade.

5. Implementação do Qdrant e Busca Vetorial

5.1 Estrutura Vetorial

Foi criada no Qdrant Cloud uma coleção denominada **reviews**, destinada ao armazenamento de embeddings gerados a partir das avaliações de produtos.

Para a vetorização dos textos foi utilizado o modelo **all-MiniLM-L6-v2**, da biblioteca Sentence Transformers, produzindo embeddings com 384 dimensões.

5.2 Processamento dos Dados

Os comentários armazenados no MongoDB foram processados por um fluxo simples de preparação dos dados, incluindo limpeza e padronização textual antes da geração dos vetores.

Os embeddings gerados foram armazenados juntamente com metadados relacionados às avaliações, permitindo consultas semânticas mais precisas.

5.3 Teste de Busca Semântica

Foi realizada uma consulta utilizando a expressão:

“produto com ótima qualidade”

O sistema retornou corretamente os documentos semanticamente mais relacionados ao significado da consulta, demonstrando a eficiência da recuperação baseada em contexto e não apenas em correspondência textual exata.

Esse resultado comprova a viabilidade da utilização de bancos vetoriais para aplicações de recomendação, análise de opinião e sistemas inteligentes de busca.

6. Segurança, Privacidade e Governança

Durante a atividade foram consideradas práticas fundamentais de segurança da informação e governança de dados.

As principais recomendações incluem:

- aplicação do princípio do menor privilégio;
- criação de usuários com permissões específicas;
- utilização de autenticação e controle de acesso;
- implementação de conexões seguras por TLS;
- realização de backups periódicos;
- monitoramento e auditoria de acessos;
- utilização de dados anonimizados em ambientes de homologação.

A solução foi desenvolvida exclusivamente com dados simulados, evitando exposição de informações pessoais reais.

Também foram observados princípios da LGPD relacionados à finalidade, necessidade, minimização de dados e rastreabilidade das operações realizadas nos sistemas.

7. Integração Inteligente dos Bancos de Dados

A arquitetura proposta utiliza cada tecnologia de acordo com sua especialidade.

O PostgreSQL é responsável pelo armazenamento estruturado de clientes, produtos e pedidos. O MongoDB armazena avaliações e comentários em formato documental. Esses textos são processados por um fluxo de transformação responsável pela geração de embeddings, posteriormente armazenados no Qdrant.

Essa integração permite que aplicações de Inteligência Artificial consultem simultaneamente:

- dados transacionais no PostgreSQL;
- avaliações e informações documentais no MongoDB;
- contexto semântico no Qdrant.

A combinação dessas tecnologias proporciona maior escalabilidade, desempenho e qualidade na recuperação das informações.

8. Utilização da IA Generativa

A IA generativa foi utilizada como ferramenta de apoio técnico durante todo o processo de análise e implementação.

As principais contribuições obtidas incluíram:

- identificação de gargalos de desempenho em consultas SQL;
- recomendação de índices para otimização;
- interpretação de planos de execução utilizando EXPLAIN ANALYZE;
- aplicação de boas práticas de modelagem;
- recomendações de segurança para MongoDB;
- definição de estratégias para integração com bancos vetoriais.

Todas as sugestões foram avaliadas, testadas e validadas antes da implementação, garantindo que a responsabilidade técnica permanecesse sob supervisão humana.

9. Conclusão

A atividade demonstrou a aplicação integrada de bancos de dados relacionais, documentais e vetoriais em um cenário moderno voltado para Inteligência Artificial.

O PostgreSQL apresentou elevada eficiência no armazenamento estruturado e na otimização de consultas por meio da utilização de índices e análise de planos de execução. O MongoDB demonstrou flexibilidade para armazenamento de avaliações e dados semiestruturados, enquanto o Qdrant possibilitou a implementação de busca semântica baseada em embeddings, ampliando as capacidades de recuperação inteligente de informações.

Os resultados obtidos evidenciam que a combinação entre PostgreSQL, MongoDB e Qdrant constitui uma arquitetura robusta, escalável e adequada para aplicações orientadas por dados e Inteligência Artificial.

Além dos ganhos de desempenho observados, a atividade reforçou a importância da segurança, da governança dos dados, da conformidade com a LGPD e do uso responsável da IA generativa como ferramenta de apoio à análise e tomada de decisão.